Split by what you ate, not equally.

A collaborative, item-by-item bill-splitting feature for **Splitwise**. One person pastes the receipt and shares a link. Everyone taps the dishes they had, and tax, tip and discounts are shared out fairly, to the paisa.

SOURCE CODE	https://github.com/chiranshee26-source/tapsplit
README / DOCS	https://github.com/chiranshee26-source/tapsplit#readme
RUN LOCALLY	`npm install` → `npm start` → http://localhost:3000
PRODUCT IMPROVED	https://www.splitwise.com

01 Product & Problem
02 My Solution
03 Visual Representation
04 Implementation
05 Working Prototype

Submitted by: chiranshee26-source

Full-stack · Node.js · Express · SQLite · Vanilla JS

1 Product & Problem

The product I picked, and what I think is broken about it

Why I picked this: every time my friends and I eat out, the bill ends in the same awkward moment. Someone says "just split it equally", the person who only had a chai quietly overpays, and the one who paid spends the next week chasing everyone for money. I use Splitwise for this, and it still doesn't fix that moment, so I wanted to.

The product: Splitwise

Splitwise is one of the most popular apps for tracking shared expenses between friends, flatmates and travel groups. It records who paid and who owes whom, and simplifies debts.

The problem: splitting a restaurant bill fairly is still painful

Splitting a restaurant bill where everyone ordered different things still works badly:

PAIN POINT	WHAT HAPPENS TODAY
"Just split it equally"	The friend who had a ₹40 chai pays the same as the one who ordered a ₹280 starter and two coffees. Equal splits are easy, so people use them, and some of them quietly feel cheated.
The payer does all the work	To split by item, one person must type every item and remember who ate what, at the table, while everyone waits.
Tax, service charge and discounts	GST, service charge and coupon discounts have to be shared out in proportion by hand. Almost nobody does this correctly, so someone always over- or under-pays.
Shared dishes	"Two of us shared the naan, three of us shared the pizza" is hard to express. People round off and guess.
Getting paid back	After the split, the payer has to chase each friend, share their UPI ID again, and keep track of who has actually paid.

Who feels it most

STUDENTS
Eat out in groups often, on tight budgets. ₹100–200 of unfairness per outing adds up.

THE PAYER
Ends up doing 10 minutes of data entry and maths, then chasing people for money for days.

LIGHT EATERS
Subsidise everyone else, or awkwardly ask to "do it properly", which slows everyone down.

Problem statement: Splitwise has no fast, fair, *collaborative* way to split one bill by what each person actually ordered. The work falls on one person, and tax, tip and discounts end up shared incorrectly.

Why existing options fall short

- **Splitwise's equal or percentage split:** fast, but not fair when orders differ.
- **Splitwise itemization:** the payer still enters and assigns every item alone, and it's limited to the payer's phone.
- **Calculator in the group chat:** error-prone, doesn't handle tax or discounts proportionally, and leaves no record.

2 My Solution: TapSplit

My idea: everyone taps their own items, instead of one person doing it all

I'm calling it **TapSplit**: a new "Split by item" flow for Splitwise. The payer pastes the receipt and shares one link. Each friend opens it on their own phone, **with no account or app needed**, and taps the dishes they had. Everyone sees the claims update live, and TapSplit works out exactly what each person owes.

How it works, in 4 steps

① PASTE
The payer pastes the receipt text (from the printed bill, an e-bill/SMS, or the phone's "copy text from photo"). A built-in parser pulls out **items, quantities, GST, service charge and discounts**, and ignores headers like "Table 7" or "Bill No".

② SHARE
TapSplit creates a short link and a **QR code**. Friends at the table scan it from the payer's phone, or the payer sends the link to the **WhatsApp** group in one tap.

③ TAP
Each friend enters their name and taps what they had. Claims appear **live on every phone** with coloured avatars. A dish tapped by several people is split evenly between them automatically.

④ SETTLE
Tax, tip and discount are shared **in proportion to what each person ordered**. Each friend gets a **"Pay via UPI"** button with the exact amount filled in, and everyone can mark when they've paid.

How the money is calculated

```
your food  = Σ (item price × qty) ÷ number of people who claimed that item
your extras = (tax + service − discount) × (your food ÷ total claimed food)
you owe   = your food + your extras
```

All amounts are stored in whole paise. Leftover paise from rounding are handed out with the *largest-remainder method*, so everyone's shares always add up **exactly** to the bill. For example, ₹100 ÷ 3 = ₹33.34 + ₹33.33 + ₹33.33.

Worked example (the sample bill in the prototype)

PERSON	WHAT THEY TAPPED	FOOD	TAX + SERVICE	DISCOUNT	PAYS
Aarav (paid)	Paneer Tikka, ½ Butter Naan, ½ Cold Coffee	₹550	₹55.00	−₹39.57	**₹565.43**
Bea	½ Butter Naan, Dal Makhani, Masala Chai	₹470	₹47.00	−₹33.81	**₹483.19**
Chen	½ Cold Coffee, Chocolate Brownie	₹370	₹37.00	−₹26.62	**₹380.38**
Total		₹1,390	₹139.00	−₹100.00	**₹1,429.00** ✓

For comparison, with an equal split everyone would pay ₹476.33. Chen would overpay by ~₹96 and Aarav would underpay by ~₹89.

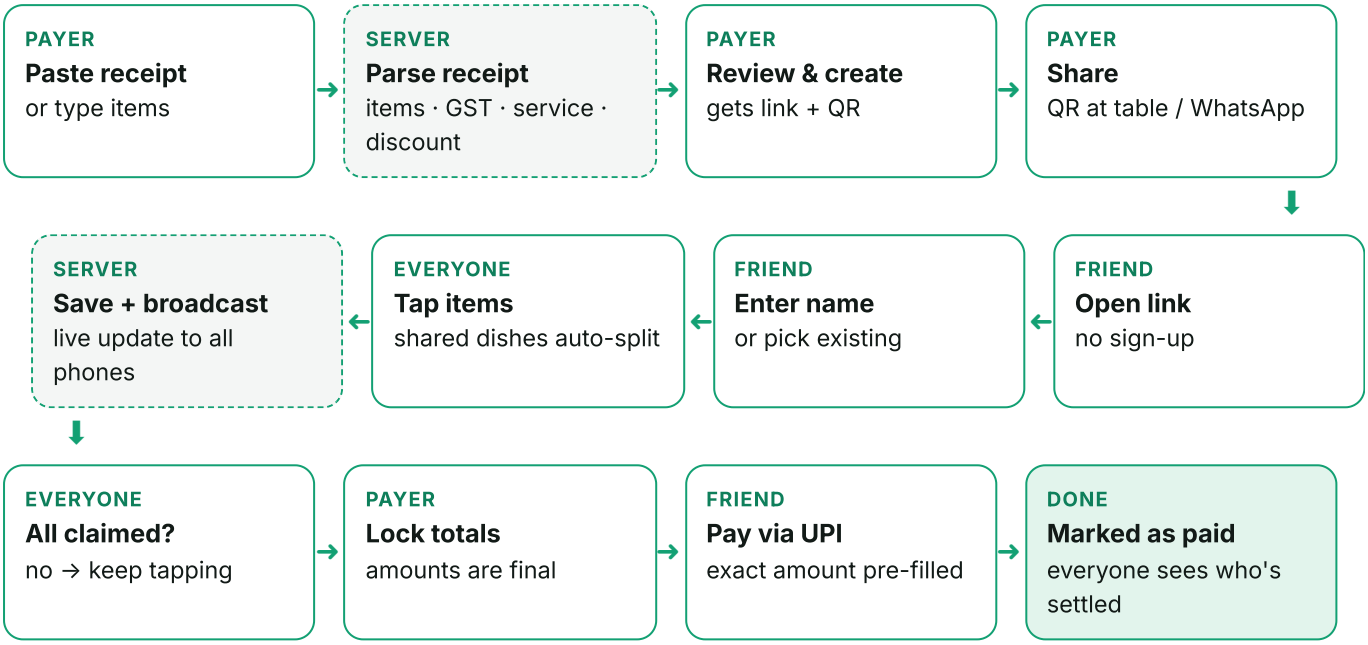
Details that make it work in real life

- **Progress bar + Unclaimed filter**, so the group knows when every item has been claimed 🎉
- **Friends without a phone:** the payer taps on their behalf.
- **Payment tracking:** "I've paid" / "Mark as received".
- **Lock totals** when done, so nobody can change them afterwards.

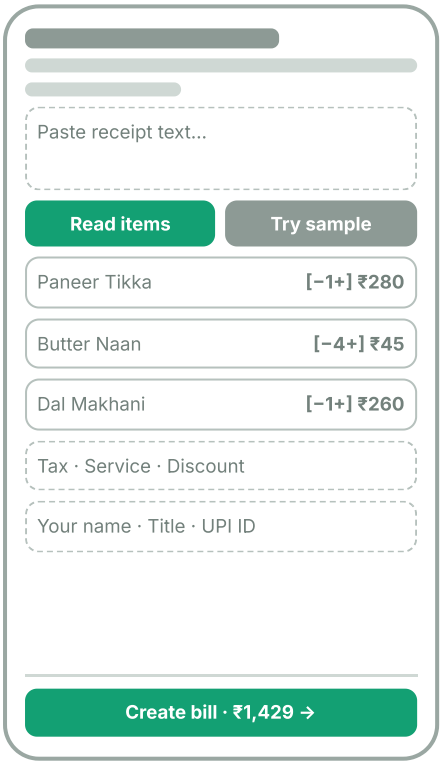
3 Visual Representation

User flow, low-fidelity wireframes, and the real UI (see the following pages)

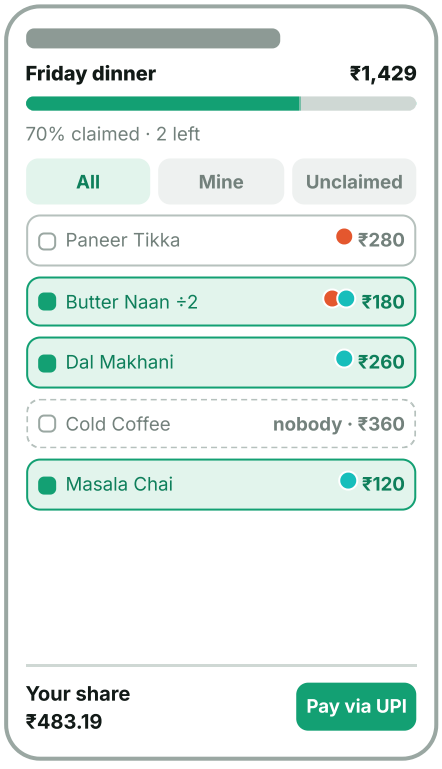
User flow



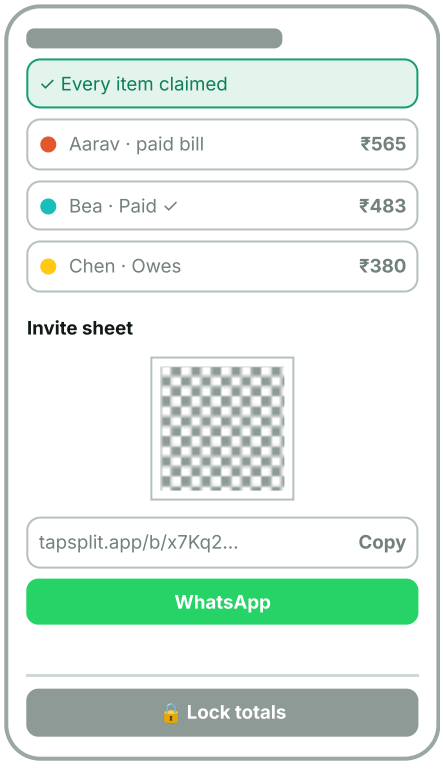
Wireframes (low fidelity)



A. Create bill



B. Tap your items (live)



C. Settle up & share

Design principles

ONE TAP = ONE ACTION

Big, thumb-friendly item cards. Tapping toggles a claim, with instant visual and haptic feedback.

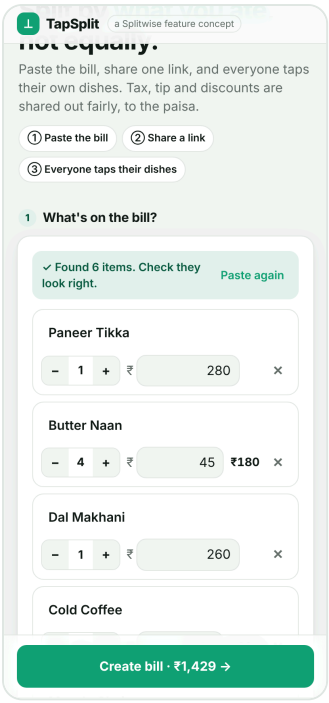
ALWAYS SHOW THE NUMBER

A sticky bottom bar always shows your running share. Tap it for the full breakdown.

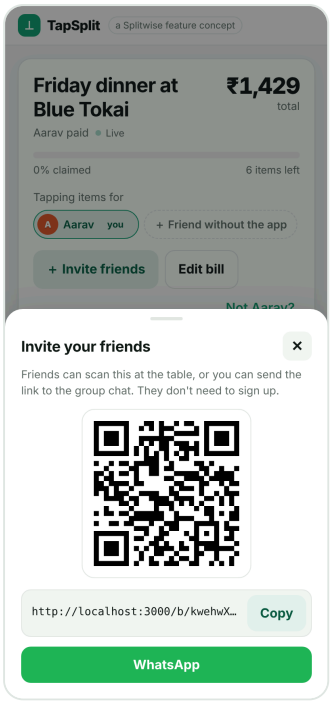
ZERO FRICTION TO JOIN

No account or app. A QR code or link, then your name, and you're in.

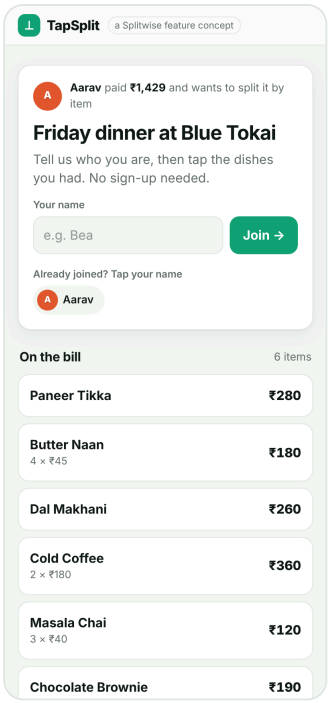
UI design: screenshots of the working prototype (mobile)



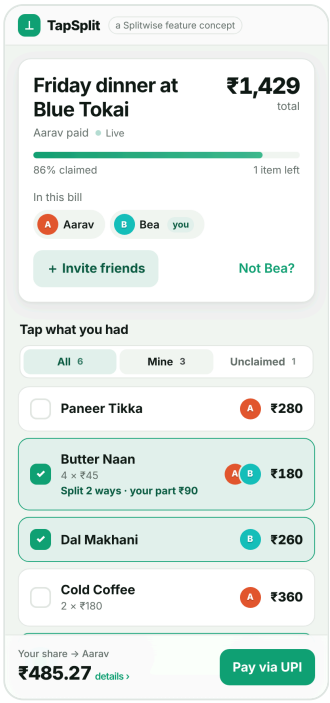
Paste → items
Receipt parsed into editable items with quantity steppers.



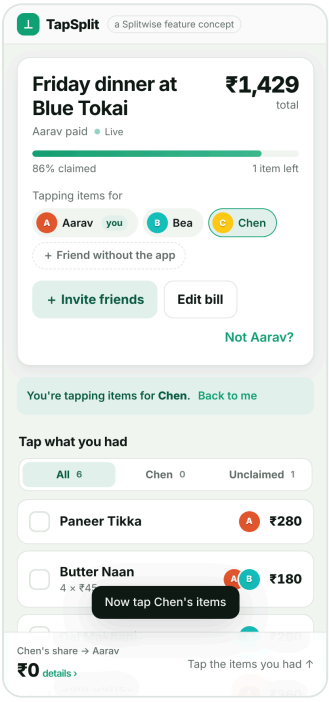
Invite sheet
QR code for the table, copy link, WhatsApp share.



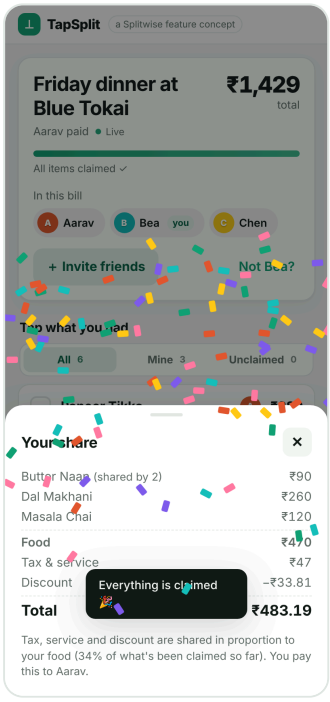
Friend joins
Opens link, types name. No sign-up.



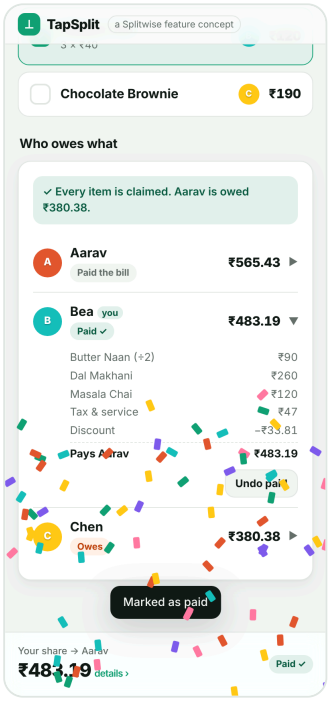
Live claiming
Avatars show who had what. Shared dishes auto-split.



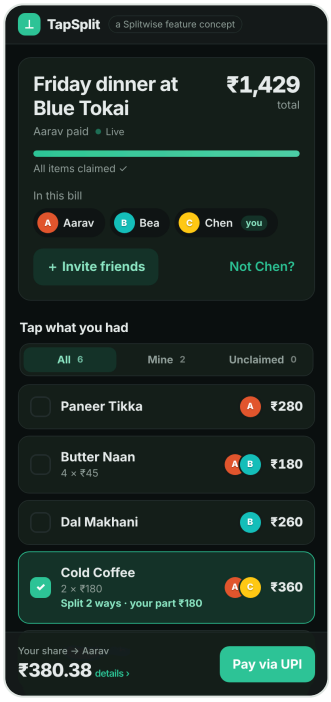
Tapping for a friend
Payer claims for someone without a phone.



Your breakdown
Every dish + proportional tax and discount. Confetti when all items are claimed.



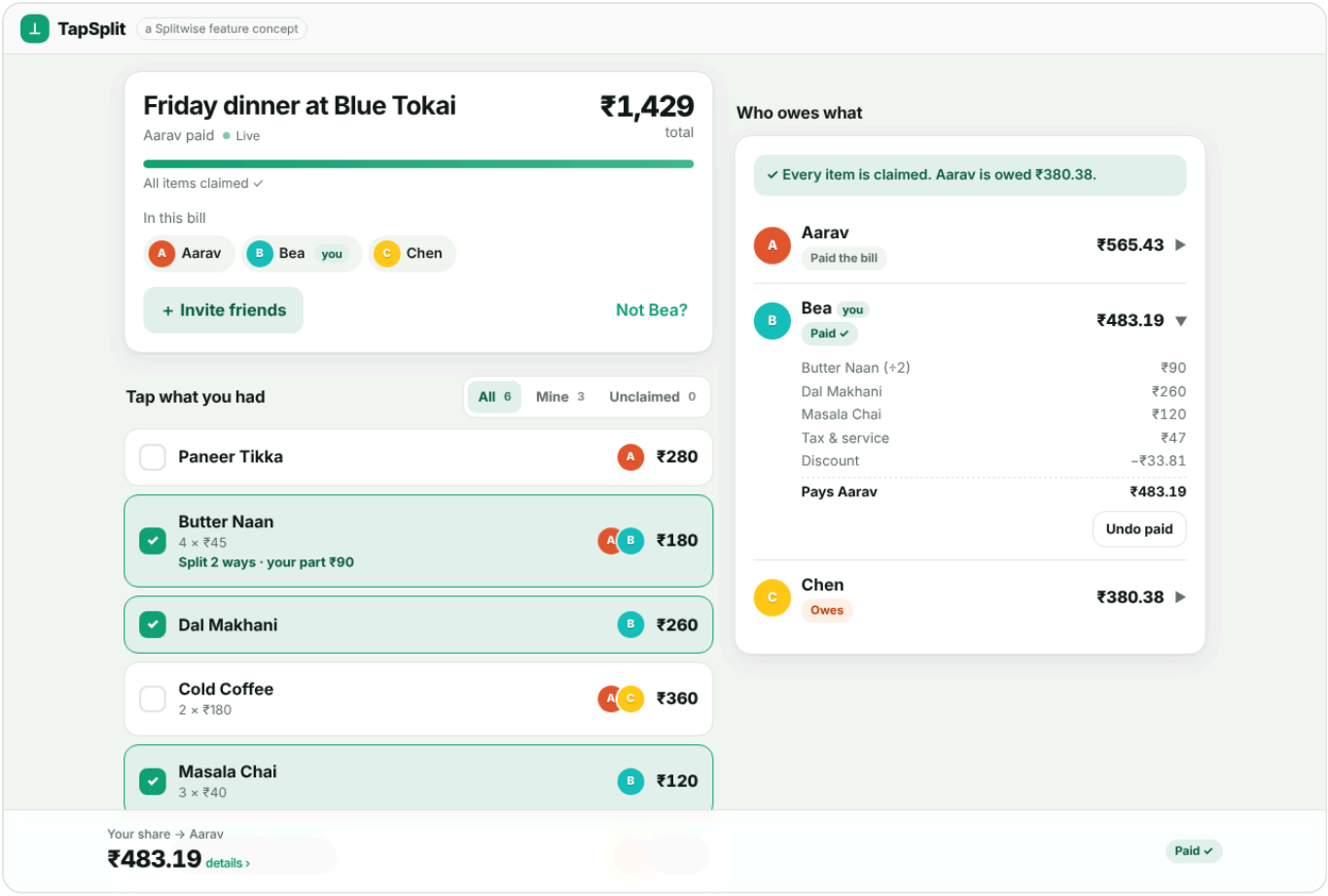
Who owes what
Per-person totals with Paid / Owes status.



Dark mode
Optional: one tap on the 🌙 toggle in the top bar.

UI design: desktop layout

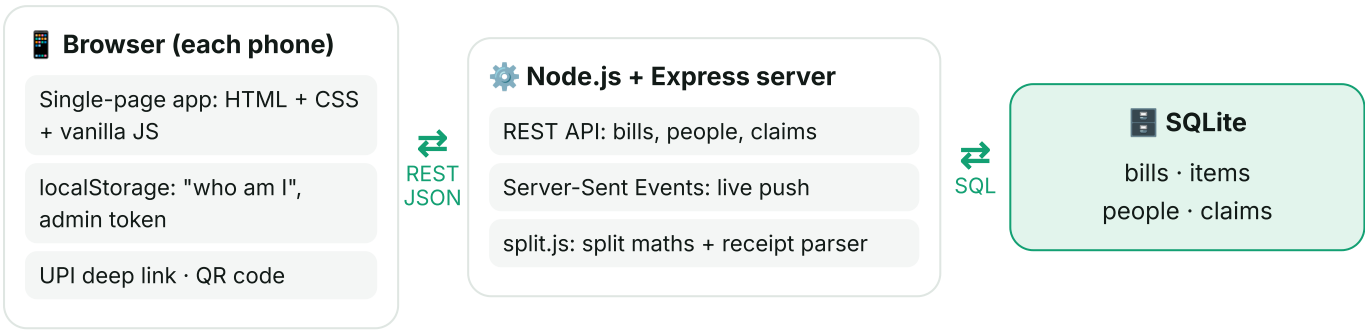
On wide screens, items are on the left and the live "Who owes what" summary stays pinned on the right.



4 Implementation

How I built it, and the tools and technologies I used

Architecture



Every claim goes to the server as a small POST request. The server saves it, recalculates the split, and pushes the new state to every phone viewing that bill over SSE.

Tech stack

LAYER	TECHNOLOGY	WHY
Frontend	HTML, CSS, vanilla JavaScript (SPA with History API routing)	No build step, loads fast on mobile, easy to read and change
Real-time	Server-Sent Events (SSE)	One-way "push the new bill state" is all that's needed. Simpler than WebSockets
Backend	Node.js 22 + Express	Small REST API, same language as the frontend
Database	SQLite via Node's built-in `node:sqlite`	Zero setup, a single file, nothing native to compile. The schema moves to PostgreSQL unchanged
Payments	UPI deep links (`upi://pay?pa=…&am=…`)	Opens GPay / PhonePe / Paytm with the amount filled in. No payment gateway needed
Sharing	`qrcode-generator` + WhatsApp share links	Friends at the same table can join in seconds
Testing	Node's built-in test runner (`node:test`)	Unit tests for the maths and parser, plus an end-to-end API test

Data model

TABLE	COLUMNS
bills	id, admin_token, title, upi_id, tax, tip, discount, payer_id, locked
items	id, bill_id, name, price, qty
people	id, bill_id, name, color, paid
claims	item_id, person_id (many-to-many)

REST API

ENDPOINT	PURPOSE
`POST /api/parse-receipt`	Text → items
`POST /api/bills`	Create bill
`GET /api/bills/:id`	Bill + split
`GET /api/bills/:id/events`	Live stream (SSE)
`POST …/people`	Join bill
`POST …/claims/toggle`	Tap an item
`POST …/people/:pid/paid`	Mark paid
`PATCH /api/bills/:id`	Payer: edit / lock

Key engineering decisions

- **Money in integer paise, never floats.** Rounding uses the largest-remainder method, so totals always match the bill exactly. This is covered by an automated test.
- **Safe without accounts.** Bill IDs are random and unguessable. Only the creator holds a secret `adminToken` for editing and locking, and locked bills reject all changes.
- **Optimistic UI.** A tapped item highlights instantly, and the live stream then confirms the server state on every phone.
- **Mobile-first UX.** Large tap targets, a sticky bottom bar, bottom sheets instead of browser pop-ups, haptics, a light/dark toggle, and support for the "reduce motion" setting.

How it would ship inside Splitwise

1. Add a **"Split by item"** button to Splitwise's *Add expense* screen that opens this flow.
2. Replace text pasting with **on-device OCR** (Google ML Kit / Apple Vision) so users simply photograph the receipt.
3. When the bill is locked, create normal **Splitwise expenses** for each person, so balances, reminders and "Settle up" keep working as they do today.
4. Scale out with **PostgreSQL** instead of SQLite and **Redis pub/sub**, so live updates work across many server instances.

Project structure

```
server.js       Express app: REST API + live updates (SSE)
db.js           SQLite schema
split.js        Split maths, rounding, receipt parser
public/         index.html · styles.css · app.js (frontend)
test/           split.test.js · api.test.js
docs/           screenshots
```

Future ideas

- Photograph a receipt → OCR fills the items
- "Your usual": suggests what you normally order there
- One settle-up for a whole trip, with the fewest transfers
- Gentle nudges to friends who haven't claimed or paid

5 Working Prototype

I built a working full-stack version: frontend, backend, database and tests

> **Source code:** https://github.com/chiranshee26-source/tapsplit
> My repository's README has this write-up, the diagrams and all the screenshots.

Run it

Requires Node.js 22.5 or newer (https://nodejs.org).

```
git clone https://github.com/chiranshee26-source/tapsplit.git
cd tapsplit
npm install
npm start       # open http://localhost:3000
npm test        # run unit + API tests
```

Try the demo in 1 minute

1. Open http://localhost:3000 and click **Try a sample bill**, type your name, and click **Create bill**.
2. Copy the link from the invite sheet and open it in a **private / incognito window**. That's "your friend". Join with a different name.
3. Tap items in both windows and watch them **update live**. Tap the same dish in both to see it split automatically.
4. Tap **Your share** for the breakdown, use **I've paid**, and, as the payer, **Lock totals**.

What's implemented

- ✔ Receipt text parser (items, qty, GST, service, discount)
- ✔ Create bill with editable items and extras
- ✔ Share via link, QR code and WhatsApp
- ✔ Join with no account (name only)
- ✔ Live multi-user claiming (Server-Sent Events)
- ✔ Shared dishes split automatically
- ✔ Proportional tax / tip / discount, exact to the paisa
- ✔ All / Mine / Unclaimed filters and progress bar
- ✔ Payer can claim for friends without a phone
- ✔ Per-person breakdown and settle-up
- ✔ UPI "Pay" deep link with exact amount
- ✔ Paid / owes tracking
- ✔ Payer-only edit, add/remove items, lock totals
- ✔ Responsive mobile + desktop, light/dark toggle

Testing

Automated tests (Node's built-in test runner) check that rounding never loses or creates a paisa (200 random cases), that shared items and tax/tip/discount are split exactly, that the receipt parser reads items and skips headers, and the full API flow: create → join → claim → paid → lock, including payer-only permissions.

Result: 5 / 5 tests passing. I also tested the UI end to end in a browser with several users at once.

What I learned

- **Money and floating point don't mix.** ₹100 ÷ 3 never adds back up, so I store everything in paise and hand out the leftover paise one at a time.
- **Live updates don't always need WebSockets.** Server-Sent Events were enough for "push the new bill to everyone", and much simpler to set up.
- **Real receipts are messy.** My first parser read "Table 7" as a ₹7 dish, so I had to teach it to skip header lines.
- **Small UX details matter.** A bottom sheet instead of a browser pop-up, or a button that always shows the total, made the app feel much better.